

PROJET INTEGRE CYBERSECURITE

Déploiement Sécurisé d'une Application Web

4 groupes, 1 seul produit, une expérience professionnelle complète

Master 1 Cybersécurité • 23 étudiants • 4 groupes de 5 • 10 jours

Niveau	Master 1 - Cybersécurité
Effectif	23 étudiants - 4 groupes de 6 personnes
Durée totale	10 jours
Outils	90 % gratuits et open source
Présentation	40 minutes - animée par chaque groupe

1. Contexte et objectif

Scenario fictif : chaque année, les étudiants de Master perdent du temps à chercher leurs conventions dans des emails, oublient des échéances de rapport, et n'ont aucune visibilité sur leur progression. StageBoard centralise tout en un seul

- **Groupe 1** - Développement : crée l'application web et la livre au Groupe 2.
 - **Groupe 2** - Infrastructure : héberge l'app sur GCP ou AWS, achète le domaine sur namecheap et configure les enregistrement DNS sur Cloudflare.
 - **Groupe 3** - Audit sécurité : teste l'app déployée et restitue un rapport de vulnérabilités.
 - **Groupe 4** - Documentation & qualité : produit toute la documentation technique et coordonne la présentation finale.
-

2. Groupe 1 — Développement de l'application web

GROUPE 1 • 6 ETUDIANTS

Développement - StageBoard

Application de suivi de stage

StageBoard est une application web de suivi de stage et d'alternance. Chaque étudiant crée un compte, renseigne son entreprise d'accueil, suit ses échéances obligatoires et tient un journal de bord.

Fonctionnalités de l'application

Tableau de bord	Vue d'ensemble : jours restants, prochaine échéance, statut rapport, contact tuteur
Suivi des échéances	Ajout et gestion des deadlines (rapport, soutenance, visite tuteur) avec statut
Journal de bord	Entrées hebdomadaires : tâches, compétences acquises, difficultés rencontrées
Fiche entreprise	Nom, secteur, adresse, coordonnées du tuteur professionnel
Compte étudiant	Inscription, connexion, profil : filière, année, type de stage ou alternance
Alertes visuelles	Rappels dans l'app quand une échéance est à moins de 7 jours

Structure : 5 pages

Page 1 — Dashboard	Vue d'ensemble après connexion : progression, prochaine échéance, dernière entrée
Page 2 — Echéances	Liste, ajout, modification des deadlines avec statut (à / en retard / termine)
Page 3 — Journal	Formulaire d'ajout d'entrées et historique complet avec filtre par date
Page 4 — Entreprise	Fiche entreprise et tuteur : infos, coordonnées, secteur d'activité
Page 5 — Auth	Inscription et connexion avec gestion de session, profil étudiant

Base de données — 3 tables

Table users	Id, nom, prénom, email, mot_de_passe, filière, année, type stage, date début, date fin
Table échéances	id, user_id, titre, description, date_limite, statut (a_venir / termine / retard)
Table journal	id, user_id, date_entree, tâches, compétences, difficultés

Mission

Le Groupe 1 est responsable de concevoir et développer une application web simple mais fonctionnelle. Elle doit être propre, versionnée et deployable par le Groupe 2 sans intervention du Groupe 1 - ce qui impose un README d'installation rigoureux.

Répartition suggérée des rôles

Etudiant 1	Chef de projet technique — coordination avec les autres groupes, suivi GitHub
Etudiant 2	Développement backend (routes, authentification, base de données)
Etudiant 3	Développement backend (formulaires, validation, sécurité des entrées)
Etudiant 4	Développement frontend (HTML, CSS, UX des pages)
Etudiant 5	Tests, débogage, rédaction du README et documentation du code

Taches détaillées

- Concevoir l'architecture de l'application (pages, base de données, flux utilisateur)
- Développer les fonctionnalités clés : page d'accueil, formulaire de contact, système login/register
- Sécuriser les entrées utilisateur (protection basique contre les injections et XSS)
- Connecter une base de données MySQL ou SQLite
- Versionner l'intégralité du code sur GitHub avec commit réguliers et messages explicites
- Rédiger un fichier README complet : prérequis, variables d'environnement, commandes d'installation
- Après réception du rapport Groupe 3 : corriger les vulnérabilités identifiées et signaler les corrections
- Collaborer avec le Groupe 4 pour la documentation du code et les captures d'écran

Stack technique conseillée

Backend	Python / Flask
Base de données	MySQL ou SQLite
Frontend	HTML, CSS, JavaScript
Versionnèment	Git + GitHub (repository partage avec G2 et G3)
Sécurité dev	Validation des entrées, paramètres prépaes SQL, CSRF ton

Livrables du Groupe 1

- Code source complet sur GitHub (repository partage avec Groupes 2, 3 et 4)
 - Fichier README avec instructions d'installation, variables d'environnement, commandes de démarrage
 - Version corrigée de l'application après rapport d'audit du Groupe 3
 - Rapport de développement : choix techniques, fonctionnalités implémentées, difficultés rencontrées
-

3. Groupe 2 — Infrastructure et déploiement

GRUPE 2 • 5 ETUDIANTS

Infrastructure & Déploiement

Héberger sur GCP ou AWS free tiers, configurer Namecheap + Cloudflare de A à Z

Mission

Le Groupe 2 prend en charge l'ensemble de la chaîne d'hébergement : création de la machine virtuelle, achat du domaine chez namecheap, configuration DNS via Cloudflare, déploiement de l'application et sécurisation du serveur. L'objectif final est que l'application soit accessible publiquement en HTTPS sur un vrai nom de domaine.

Répartition suggérée des rôles

Etudiant 1	Chef infra — coordination avec G1 (réception du code) et G4 (documentation)
Etudiant 2	Administration GCP : création VM, règles pare-feu, gestion SSH
Etudiant 3	Achat domaine Namecheap, configuration DNS, intégration Cloudflare
Etudiant 4	Installation et configuration Nginx, déploiement de l'application
Etudiant 5	Sécurisation serveur : UFW, Fail2ban, tests de connectivité, monitoring

Taches détaillées

Etape 1 — Création de la VM sur Google Cloud

- Créer un compte Google Cloud et activer le Free Tier (e2-micro, Ubuntu 22.04 LTS)
- Configurer l'accès SSH avec paire de clés RSA (désactiver authentification par mot de passe)
- Ouvrir les ports 80 (HTTP) et 443 (HTTPS) dans les règles de pare-feu GCP

Etape 2 — Achat du nom de domaine sur Namecheap

- Choisir un nom de domaine
- Pointer les serveurs DNS du domaine vers Cloudflare (remplacer les Name servers Namecheap)

Etape 3 — Configuration Cloudflare

- Ajouter le domaine dans Cloudflare, créer un enregistrement A pointant vers l'IP de la VM
- Activer le proxy Cloudflare (icône nuage orange) pour masquer l'IP réelle du serveur
- Configurer le mode SSL/TLS en Full et activer la redirection automatique HTTP vers HTTPS
- Activer les règles WAF de base disponible sur le plan gratuit (protection bots, rate limitent)

Etape 4 — Déploiement de l'application

- Installer Nginx et configurer le Virtual host pour le domaine acheté
-

- Cloner le repository GitHub du Groupe 1 et suivre le README d'installation
- Installer les dépendances (Python, base de données), configurer les variables d'environnement
- Installer UFW : autoriser uniquement les ports 22, 80 et 443
- Installer et configurer Fail2ban pour protéger SSH et Nginx contre la brute force

Outils utilisés

Cloud	Google Cloud Platform Free Tier — VM e2-micro-Ubuntu
Domaine	Namecheap
CDN / DNS / WAF	Cloudflare (plan gratuit)
Serveur web	Nginx
Sécurité serveur	UFW + Fail2ban
Accès distant	SSH avec clés RSA (authentification par mot de passe désactivée)

Livrables du Groupe 2

- Application accessible en HTTPS sur le domaine acheté — URL communiquée aux Groupes 3 et 4
- Capture d'écran du Dashboard Cloudflare : enregistrements DNS, mode SSL, WAF active
- Rapport de configuration : VM GCP, Nginx, UFW, Fail2ban, Cloudflare (avec justifications)
- Fichier de configuration Nginx anonymisé joint en annexe

4. Groupe 3 — Audit et analyse de vulnérabilités

GRUPE 3 • 6 ETUDIANTS

Audit de Sécurité

Scanner, analyser et restituer les vulnérabilités de l'application déployée

Mission

Le Groupe 3 intervient en tant qu'équipe de sécurité externe. Il réalise un audit complet de l'application déployée par le Groupe 2, classe les vulnérabilités par criticité et restitue ses résultats de manière professionnelle au Groupe 1 pour correction. Un pré-test est effectué après corrections.

Répartition suggérée des rôles

Etudiant 1	Chef audit — coordination avec G1 (réception corrections) et G4 (remise rapport)
Etudiant 2	Scan automatique : OWASP ZAP et Niko — configuration et interprétation des résultats
Etudiant 3	Tests manuels : formulaires, authentification, XSS basique, injection SQL
Etudiant 4	Analyse SSL Labs, en-têtes HTTP (securityheaders.com), vérification fichiers exposés
Etudiant 5	Rédaction du rapport d'audit, classification CVSS, re-test après corrections

Taches détaillées

Phase 1 — Reconnaissance et cartographie

- Identifier la surface d'attaque : pages exposées, formulaires, Endpoint, technologies utilisées
- Vérifier les en-têtes HTTP de sécurité avec securityheaders.com (X-Frame-Options, CSP, HSTS...)
- Analyser la configuration SSL/TLS avec SSL Labs — objectif : note A ou A+
- Vérifier l'absence de fichiers sensibles exposés : env., git, config. PHP, backup. SQL

Phase 2 — Scan automatique

- Lancer un scan complet avec OWASP ZAP en mode automatique sur l'URL de l'application
- Effectuer un scan complémentaire avec Niko pour détecter les mauvaises configurations serveur
- Identifier les vulnérabilités OWASP Top 10 : injections SQL, XSS, CSRF, exposition de données

Phase 3 — Tests manuels

- Tester les formulaires : tentatives d'injection SQL basiques et XSS réfléchis
 - Vérifier le mécanisme d'authentification : politique mots de passe, gestion session, protection brute force
 - Contrôler les messages d'erreur : ne doivent pas divulguer de détails techniques (stack traces, chemins)
-

Phase 4 — Restitution et pré-test

- Rédiger le rapport d'audit : chaque vulnérabilité classée par criticité (Critique / Haute / Moyenne / Faible)
- Transmettre le rapport au Groupe 1 pour correction ET au Groupe 4 pour intégration dans la synthèse
- Réaliser un pré-test après corrections et documenter les résultats (failles corrigées vs. Restantes)

Outils utilisés

Scanner web	OWASP ZAP (Zed Attack Proxy) — gratuity
Scanner serveur	Niko
Tests manuels	Burg Suite Community Edition
Audit SSL/TLS	SSL Labs (ssllabs.com)
Audit en-têtes	securityheaders.com
Référentiel	OWASP Top 10 — classification CVSS

Livrables du Groupe 3

- Rapport d'audit complet : description de chaque vulnérabilité, criticité, captures d'écran, preuve
- Tableau synthétique des vulnérabilités (nom, type, criticité, statut après correction)
- Rapport de pré-test final après corrections du Groupe 1
- Recommandations générales pour sécuriser durablement l'application

5. Groupe 4 — Documentation et qualité projet

GRUPE 4 • 5 ETUDIANTS

Documentation & Qualité

Produire la documentation complète et coordonner la présentation finale

Mission

Le Groupe 4 joue un rôle transversal : il interagit avec les trois autres groupes tout au long du projet pour collecter l'information et produire une documentation professionnelle complète. Il coordonne également la présentation finale des quatre groupes. Ce rôle correspond en entreprise à un profil Tech Writer ou Chef de projet technique.

Répartition suggérée des rôles

Etudiant 1	Chef doc & coordinateur — liaison avec G1, G2, G3 ; suivi du planning général
Etudiant 2	Guide utilisateur : captures d'écran, scénarios d'usage, FAQ
Etudiant 3	Documentation technique : schéma réseau (Draw.io), flux de données, stack complète
Etudiant 4	Guide de maintenance infra : procédures mises à jour, sauvegardes, surveillance
Etudiant 5	Synthèse projet + préparation et animation de la présentation finale (slides, timing)

Taches détaillées

Collecte continue

- Participer aux points de synchronisation inter-groupes et prendre les notes de réunion
- Collecter les configurations, captures d'écran et informations techniques auprès des Groupes 2 et 3
- Suivre l'avancement du projet et signaler les retards au formateur si nécessaire

Guide utilisateur

- Rédiger un guide complet pour un utilisateur final non technique de l'application
- Inclure des captures d'écran annotées de chaque fonctionnalité (inscription, connexion, formulaire)
- Ajouter une section FAQ avec les questions fréquentes

Documentation technique

- Créer le schéma d'architecture réseau complet : VM GCP, Cloudflare, Nginx, base de données (Draw.io)
 - Documenter le flux de données : de la requête utilisateur jusqu'à la réponse serveur
 - Lister l'ensemble du stack technique avec versions et liens de documentation officielle
-

Guide de maintenance

- Documenter la procédure de mise à jour du serveur et de l'application
- Décrire la procédure de sauvegarde de la base de données
- Expliquer comment consulter les logs Nginx et Fail2ban pour détecter une anomalie
- Lister les commandes essentielles pour redémarrer les services en cas d'incident

Outils utilisés

Schémas	Draw.io (diagrams.net) — gratuit en ligne
Documentation	Word / Google Docs / Notion
Présentation	PowerPoint / Google Slides / Canvas
Communication	WhatsApp groupe / email / Discord selon préférence
Suivi planning	Tableau Kanban simple (Trello gratuit ou tableau blanc)

Livrables du Groupe 4

- Guide utilisateur de l'application (format PDF, minimum 10 pages avec captures)
- Schema d'architecture technique complet et annoté (Draw.io ou équivalent)
- Guide de maintenance de l'infrastructure (procédures claires et testées)
- Document de synthèse projet : résumé des travaux des 4 groupes avec tableau bilan

7. Critères d'évaluation

Chaque groupe est évalué sur ses propres livrables.

Critère	Poids	Description
Qualité technique du livrable	40 %	Fonctionnalité, configuration, sécurité effective
Documentation et rapport écrit	25 %	Clarté, rigueur, captures d'écran, structure
Présentation orale et démo live	20 %	Clarté, maîtrise du sujet, réponse aux questions
Collaboration inter-groupes	15 %	Communication, respect des délais, adaptabilité

8. Ressources et liens utiles

Google Cloud Free Tier	cloud.google.com/free — VM e2-micro permanente
Namecheap	namecheap.com
Cloudflare	cloudflare.com — plan gratuit suffisant
OWASP ZAP	zapproxy.org — scanner web gratuit
SSL Labs	ssllabs.com/Ss test — audit SSL/TLS
Security Headers	securityheaders.com — audit end-tetes HTTP
OWASP Top 10	owasp.org/Top10 — referential vulnerabilities
Draw.io	diagrams.net — schémas d'architecture gratuits
Fail2ban	fail2ban.org — protection brute force

Bonne chance aux quatre équipes !

Ce projet est une opportunité unique de travailler en équipe sur une infrastructure réelle, du code au déploiement jusqu'à l'audit de sécurité.
